

Tribhuvan University
Institute of Engineering
Purwanchal Campus
पूर्वाञ्चल क्याम्पस

Lab Report 3

System Architecture, Use Case, Activity, Sequence, Component, and Class Diagram Design

Project:

Nepali Lifestyle Based Diet and Fitness Advisory System

Course Software Engineering (Practical)

Course Code ENCT 352

Submitted By

HIMAL BHANDARI	PUR080BCT034
KSHITIZ PORTEL	PUR080BCT039
MANAV ACHARYA	PUR080BCT041
MAUSAM PARAJULI	PUR080BCT045

Submitted To Department of Electronics and Computer Engineering

Instructor Asst. Prof. Binay Lal Shrestha

Date July 21, 2026

Contents

1 Introduction **2**

1.1 Architectural Block Diagram . . . 3

1.1.1 Major Components . . . 3

1.1.2 Relationships and Workflow . . . 4

1.1.3 Design Discussion . . . 4

1.1.4 Summary . . . 4

2 Behavioral Modeling **6**

2.1 Use Case Model . . . 6

2.1.1 Major Actors and Use Cases . . . 6

2.1.2 Include and Extend Relationships . . . 7

2.1.3 Design Discussion . . . 7

2.1.4 Summary . . . 7

2.2 Activity Workflow . . . 8

2.2.1 Major Activity Nodes and Swimlanes . . . 8

2.2.2 Workflow Description . . . 8

2.2.3 Design Discussion . . . 9

2.2.4 Summary . . . 9

3 Structural and Dynamic Interaction Modeling **10**

3.1 Sequence Model . . . 10

3.1.1 Major Participants (Lifelines) . . . 10

3.1.2 Interaction Sequence Rationale . . . 11

3.1.3 Design Discussion . . . 11

3.1.4 Summary . . . 11

3.2 Component Model . . . 12

3.2.1 Major Components and Interfaces . . . 12

3.2.2 Component Dependencies and Interfaces . . . 13

3.2.3 Design Discussion . . . 13

3.2.4 Summary . . . 13

4 Object-Oriented Static Design **14**

4.1 Class Diagram Model . . . 14

4.1.1 Major Classes and Attributes . . . 14

4.1.2 Relationships and Multiplicities . . . 15

4.1.3 Design Discussion . . . 15

4.1.4 Summary . . . 16

5 Overall Discussion **17**

6 Conclusion **18**

1 Introduction

This laboratory report presents the comprehensive behavioral, structural, and static object-oriented UML design for the Nepali Lifestyle Based Diet and Fitness Advisory System. Building upon the requirements analysis established in Lab 1 and the Data Flow Diagrams (DFD) developed in Lab 2, this phase models the system's runtime dynamics, component interactions, static domain entities, and multi-tier architectural layout. While DFDs describe data movement between abstract processes and persistent stores, UML models capture object-oriented structure, execution control, domain relationships, and component boundaries necessary for software construction.

Together, these models bridge the gap between high-level functional specifications and low-level software implementation. By specifying clear boundaries across client applications, gateway routers, analytical processing engines, static domain classes, database storage, and external artificial intelligence services, this document establishes a maintainable foundation for subsequent database schema creation, API routing, and class implementation.

1.1 Architectural Block Diagram

Figure 1.1 illustrates the multi-tier architectural block diagram of the system, defining core subsystems, service boundaries, and data paths required to deliver dietary analysis and personalized health recommendations.

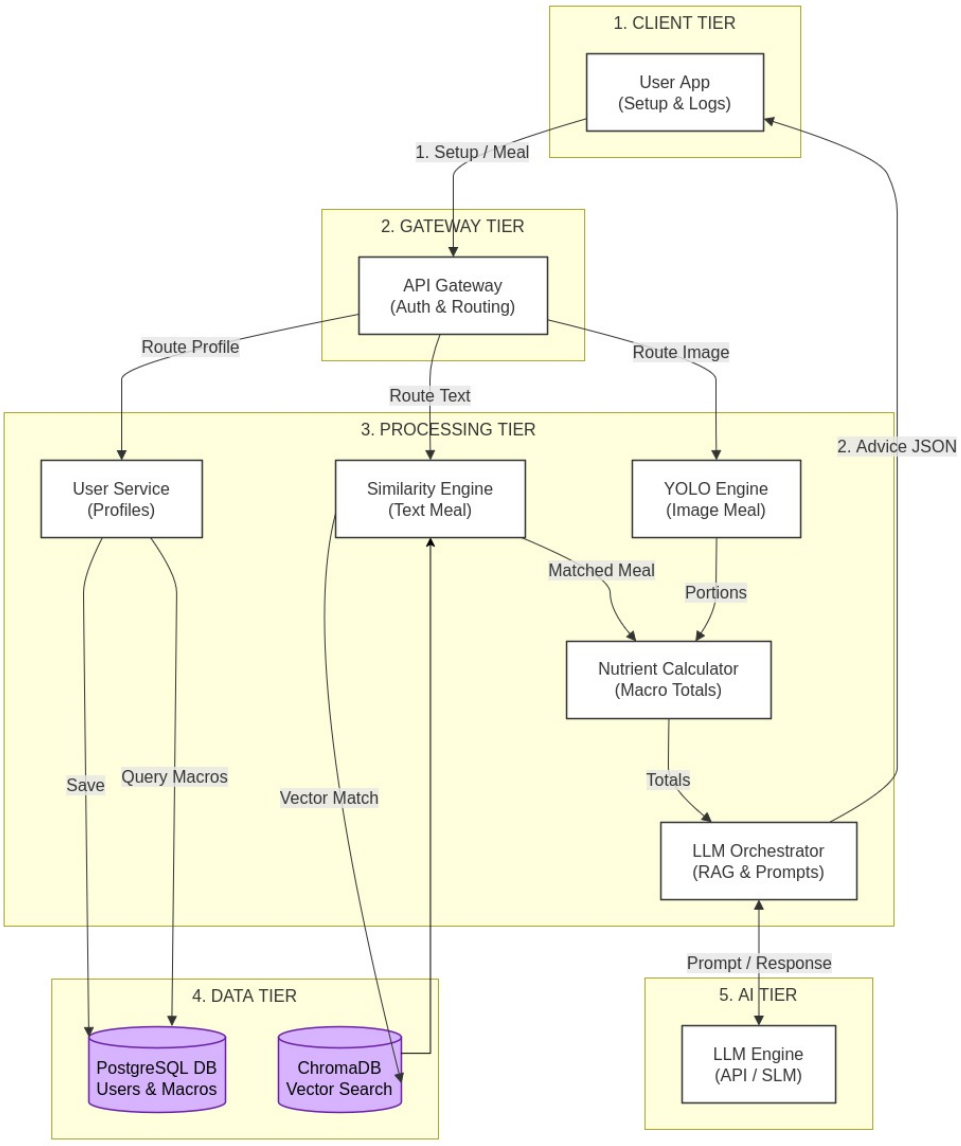


Figure 1.1: Block Diagram of System Architecture

1.1.1 Major Components

The system architecture is structured into five distinct operational tiers:

1. **Client Tier**: Contains the User App interface responsible for capturing user profile parameters, health metrics, and meal inputs (textual descriptions or image uploads), as well as rendering generated advisory reports.

2. **Gateway Tier**: Hosts an API Gateway that acts as the single point of entry, providing request routing, authentication verification, and workload distribution to downstream

microservices.

3. **Processing Tier**: Encompasses core business logic services, including the User Service (profile management), Similarity Engine (vector-based textual meal search), YOLO Engine (computer vision object detection for food items), Nutrient Calculator (macronutrient aggregation), and LLM Orchestrator (Retrieval-Augmented Generation context assembly).

4. **Data Tier**: Comprises a PostgreSQL relational database for structured user profile and macro history storage, alongside a ChromaDB vector database for semantic dish matching and similarity retrieval.

5. **AI Tier**: Contains the Large Language Model (LLM) engine (accessed via external API or local SLM) that synthesizes contextualized dietary and fitness recommendations.

1.1.2 Relationships and Workflow

The workflow initiates at the Client Tier when a user submits profile updates or meal data. The API Gateway inspects incoming HTTP/REST requests, validates session credentials, and routes traffic to the appropriate service in the Processing Tier. Textual meal descriptions are directed to the Similarity Engine, which queries ChromaDB for vector matches, whereas image uploads are routed to the YOLO Engine for automated food recognition and portion estimation. Identified food items are forwarded to the Nutrient Calculator to aggregate daily caloric and macronutrient totals. These totals, combined with user medical constraints from PostgreSQL, are fed into the LLM Orchestrator. The orchestrator constructs a structured prompt for the LLM Engine, which generates personalized advice and returns it through the Gateway to the user interface.

1.1.3 Design Discussion

The multi-tier design adheres strictly to the separation of concerns principle. By isolating vision processing (YOLO Engine) and text matching (Similarity Engine) from user management (User Service), individual services can scale independently based on operational demand. Utilizing a hybrid database strategy—pairing PostgreSQL for ACID-compliant transactional state with ChromaDB for high-dimensional vector search—ensures both data integrity for user health records and fast retrieval performance for regional food item embeddings. Furthermore, encapsulating the LLM Engine behind an orchestration service decouples core application business logic from specific AI model vendor implementations, enabling seamless model upgrades or local SLM integration without modifying core application code.

1.1.4 Summary

The architectural block diagram establishes the physical and logical boundaries of the platform. It verifies that complex multimodal processing (image, text, and generative AI) can be

executed cleanly through modular service interactions, serving as the blueprint for component specification and network deployment.

2 Behavioral Modeling

Behavioral modeling captures the functional capabilities of the system from both external actor perspectives and internal operational workflows. The Use Case Diagram identifies system boundaries and user capabilities, while the Activity Diagram details procedural flow and decision logic during execution.

2.1 Use Case Model

Figure 2.1 presents the Use Case Diagram for the Nepali Lifestyle Based Diet and Fitness Advisory System, illustrating actors and discrete functional units supported by the application.

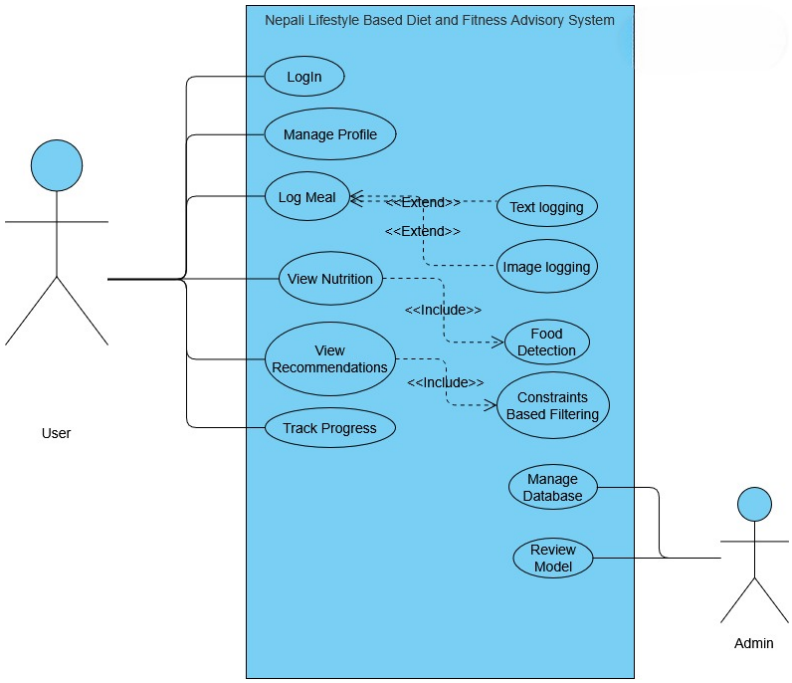


Figure 2.1: Use Case Diagram of the Advisory System

2.1.1 Major Actors and Use Cases

Two primary actors interact with system boundaries:

- **User**: The primary end-user who registers on the platform to track daily dietary intake, view nutritional breakdowns, receive personalized fitness guidance, and monitor long-term health metrics. The User interacts directly with six core use cases: *LogIn*, *Manage Profile*, *Log Meal*, *View Nutrition*, *View Recommendations*, and *Track Progress*.
- **Admin**: The administrative actor responsible for platform maintenance and operational governance. The Admin interacts with two system management use cases: *Manage Database* (curating food nutritional records) and *Review Model* (monitoring recognition accuracy and recommendation alignment).

2.1.2 Include and Extend Relationships

The model incorporates explicit relationship semantics to organize behavioral dependencies:

- **Extend Relationships**: The base *Log Meal* use case is extended by two optional interaction pathways: $<<$ Extend $>>$ *Text Logging* and $<<$ Extend $>>$ *Image Logging*. This captures user choice at runtime without cluttering the primary meal logging specification.
- **Include Relationships**: Mandatory functional dependencies are modeled using $<<$ Include $>>$. The *View Nutrition* use case mandatory includes $<<$ Include $>>$ *Food Detection*, ensuring nutritional values cannot be rendered without underlying item identification. Similarly, *View Recommendations* mandatory includes $<<$ Include $>>$ *Constraints Based Filtering*, guaranteeing that generated advice is filtered against user medical conditions and cultural constraints prior to presentation.

2.1.3 Design Discussion

The design enforces high cohesion by grouping user capabilities into distinct functional goals while separating operational administration from end-user workflows. Using extend relationships prevents conditional branching clutter inside the base *Log Meal* specification, adhering to the Open-Closed Principle where new input modalities can be added without altering core meal logging logic. Mandatory inclusion of constraint filtering reflects a safety-critical design choice: dietary recommendations must undergo deterministic rule validation before reaching the user, preventing unsafe suggestions for users with medical conditions such as diabetes or hypertension.

2.1.4 Summary

The Use Case Diagram defines the complete scope of user-facing and administrative requirements. It provides the functional reference against which user interface routes, access control matrices, and API authorization policies are developed.

2.2 Activity Workflow

Figure 2.2 depicts the Activity Diagram for the meal entry, nutritional processing, and advice generation workflow, organized across three functional swimlanes: Users, Web Application, and Database / YOLO Model.

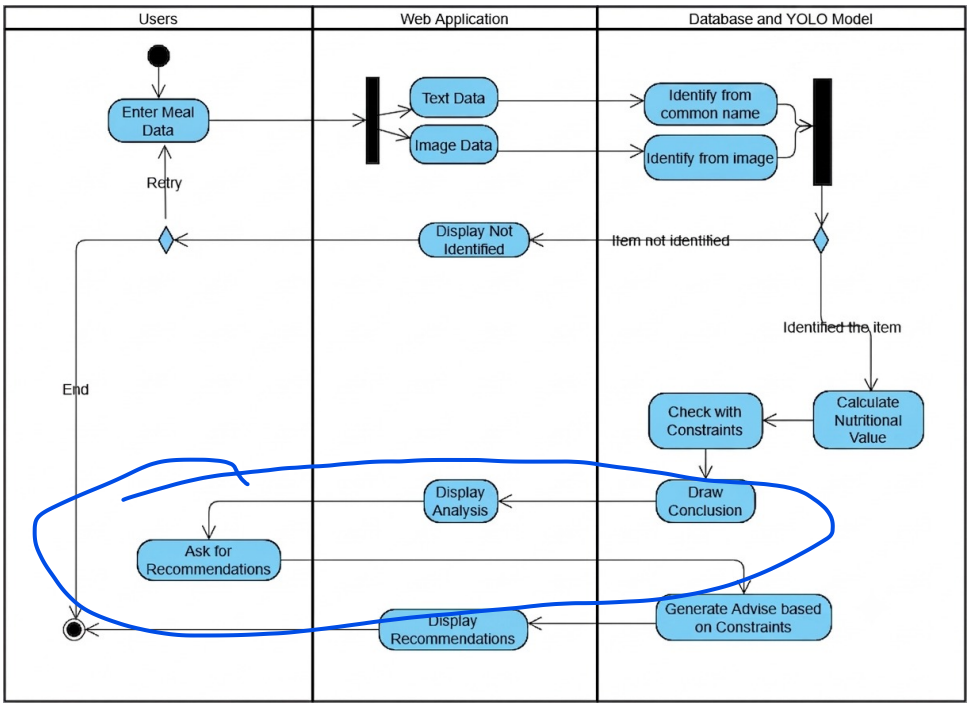


Figure 2.2: Activity Diagram for Meal Logging and Recommendation Workflow

2.2.1 Major Activity Nodes and Swimlanes

The process flow is partitioned into three swimlanes reflecting responsibility boundaries:

- **Users Swimlane**: Captures user-initiated triggers, including entering meal data, selecting recommendation requests, and terminal decision branches (retrying input or exiting).
- **Web Application Swimlane**: Handles client-side request validation, input formatting (splitting text versus image payloads), display of nutrient analysis, and final recommendation presentation.
- **Database and YOLO Model Swimlane**: Represents backend computational nodes responsible for text-based common name identification, computer vision image detection, nutrient aggregation, constraint validation, and LLM advice generation.

2.2.2 Workflow Description

The activity begins when a user inputs meal data. The Web Application receives the payload and forks the execution path depending on whether the input is text or image format. Backend components attempt item identification via text matching or YOLO vision model inference. A

decision diamond evaluates identification success: if no food item is recognized, an error state is returned to the user, offering a retry option or workflow exit. Upon successful identification, the system calculates the meal's nutritional values, evaluates medical and dietary constraints, and displays the nutritional breakdown to the user. If the user requests advice, the backend generates constraint-aware recommendations and displays them on the user interface before terminating the process.

2.2.3 Design Discussion

The activity flow demonstrates robust error handling and defensive design. By inserting an explicit validation decision point immediately after food recognition, the system prevents execution of downstream nutrient calculations or LLM prompt generation when input data is invalid. Swimlane partitioning reinforces architectural separation of concerns, ensuring that client-side display logic remains decoupled from backend algorithm execution and persistent storage operations.

2.2.4 Summary

The Activity Diagram clarifies operational execution sequences, decision logic, and exception recovery paths for meal logging. It forms the algorithmic foundation for designing client-side state machines and backend middleware handlers.

3 Structural and Dynamic Interaction Modeling

Structural and dynamic interaction modeling details how system components collaborate at runtime to fulfill end-to-end scenarios and how software artifacts are organized into deployable modules.

3.1 Sequence Model

Figure 3.1 illustrates the Sequence Diagram for an end-to-end user scenario comprising user registration, authentication, meal image upload, nutritional calculation, and recommendation synthesis.

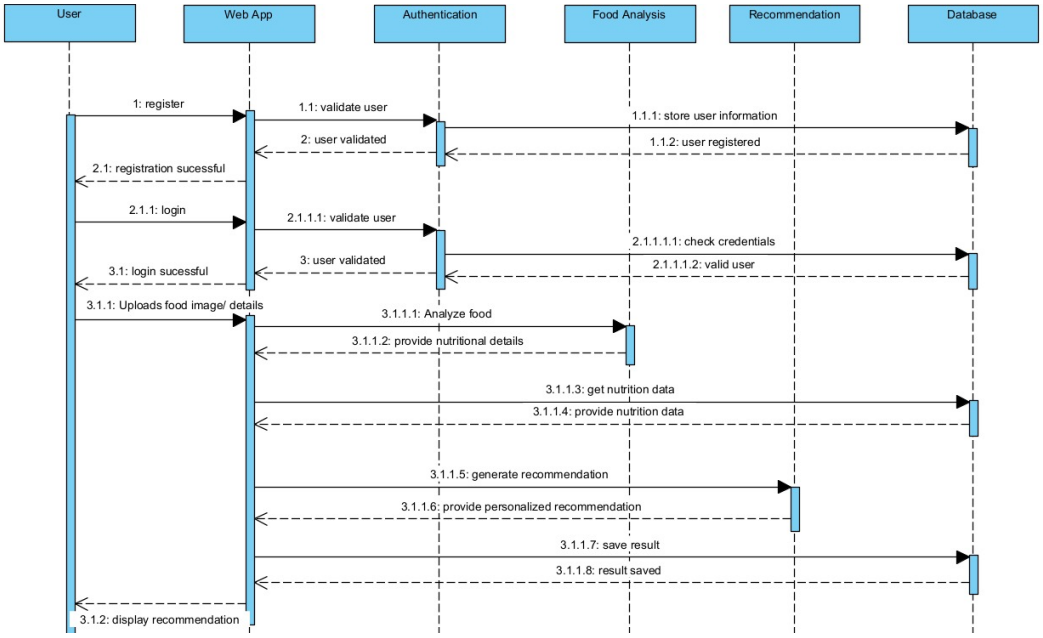


Figure 3.1: Sequence Diagram for Registration, Login, and Meal Recommendation

3.1.1 Major Participants (Lifelines)

The interaction involves six key lifelines operating across system boundaries:

- **User**: The human actor initiating requests.
- **Web App**: The central client/frontend orchestrator managing user interactions.
- **Authentication**: The security module validating user credentials and session tokens.
- **Food Analysis**: The analytical processing engine running vision detection and nutritional lookup.
- **Recommendation**: The AI advisory engine synthesizing contextual fitness and diet recommendations.
- **Database**: The persistent relational and vector data store.

3.1.2 Interaction Sequence Rationale

The scenario progresses through three distinct operational phases:

1. **Registration and Authentication Phase**: The User submits registration details to the Web App, which delegates credential validation and user persistence to the Authentication module and Database. Following registration, the user authenticates via a login call, receiving session validation.

2. **Meal Analysis Phase**: The authenticated User uploads meal details to the Web App. The request is forwarded to the Food Analysis module, which identifies food items, calculates nutritional metrics, queries the Database for profile baselines, and returns a verified nutrition summary.

3. **Recommendation and Persistence Phase**: The Web App forwards the complete nutritional profile to the Recommendation module. The Recommendation module synthesizes a personalized diet plan, persists the result in the Database, and returns the response through the Web App to the User interface.

3.1.3 Design Discussion

The sequence design emphasizes loose coupling and secure communication boundaries. The Web App acts as a mediator, ensuring the end-user interface never communicates directly with backend storage or AI analytical modules. Hierarchical message numbering $(1.1, 1.1.1)$ clarifies synchronous call chains and return paths, establishing explicit transactional boundaries. Security is maintained by requiring session authentication prior to invoking computationally intensive food analysis and recommendation services.

3.1.4 Summary

The Sequence Diagram validates that user-facing requirements can be realized through a precise, time-ordered sequence of inter-module messages. It serves as the blueprint for defining RESTful API endpoints, request/response payload schemas, and backend service methods.

3.2 Component Model

Figure 3.2 presents the Component Diagram of the system architecture, illustrating physical software components, interface boundaries, and physical database drivers across the system's five operational tiers.

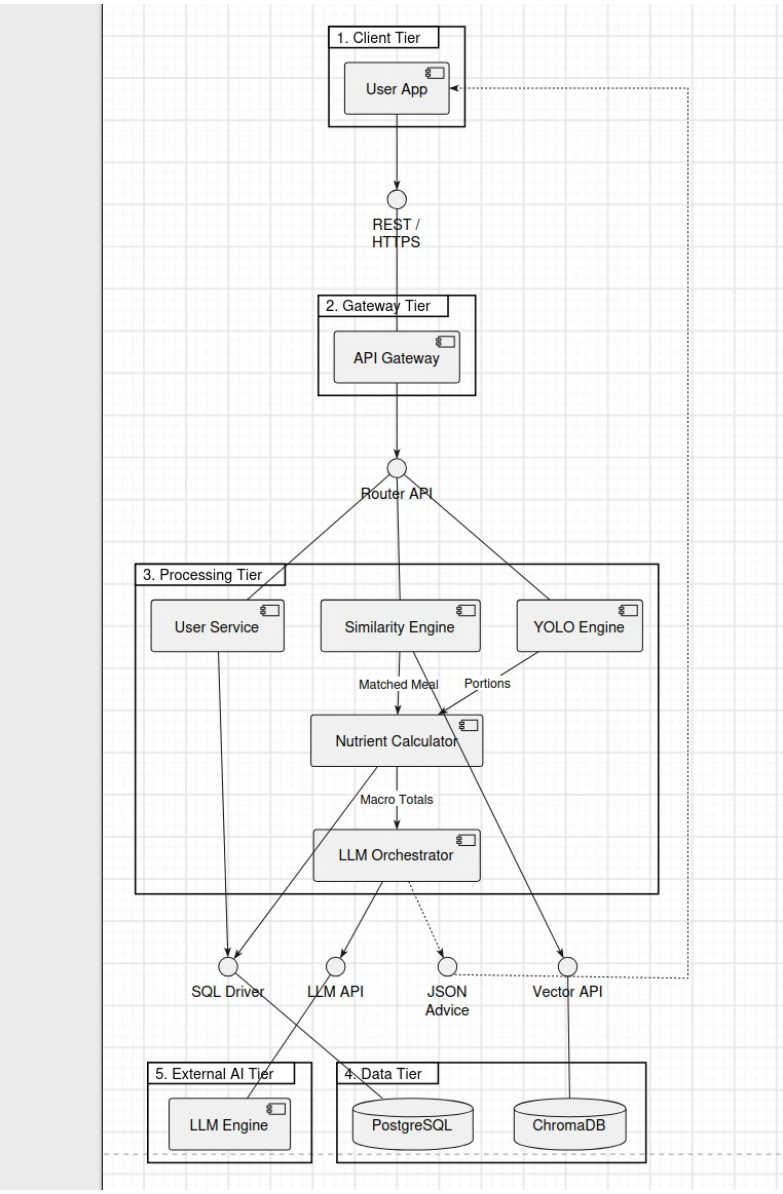


Figure 3.2: Component Diagram of System Architecture

3.2.1 Major Components and Interfaces

The component model details deployable software modules and exposed interfaces:

- **Client Tier**: Contains the *User App* component exposing user interface views and connecting via a REST/HTTPS interface.
- **Gateway Tier**: Contains the *API Gateway* component providing the Router API interface for downstream processing services.

- **Processing Tier**: Encompasses modular logic components—*User Service*, *Similarity Engine*, *YOLO Engine*, *Nutrient Calculator*, and *LLM Orchestrator*—which intercommunicate via internal service calls and interface drivers.
- **Data Tier & External AI Tier**: Connects storage components (*PostgreSQL* and *ChromaDB*) via SQL Driver and Vector API interfaces, while interfacing with the *LLM Engine* via an external LLM API.

3.2.2 Component Dependencies and Interfaces

Component dependencies follow strict interface contracts. The User App communicates exclusively with the API Gateway over HTTPS. The API Gateway routes requests through the Router API to specific processing components. The Similarity Engine depends on the Vector API to query ChromaDB, while the User Service utilizes the SQL Driver to interact with PostgreSQL. The LLM Orchestrator aggregates data from the Nutrient Calculator, queries user constraints via the SQL Driver, and dispatches formatted requests through the LLM API to the external AI Tier, returning JSON-formatted advice back to the client.

3.2.3 Design Discussion

The Component Diagram demonstrates high modularity and adherence to Interface Segregation principles. Defining explicit interface sockets (REST/HTTPS, Router API, SQL Driver, Vector API) ensures that internal implementations of individual components can be modified or replaced without breaking dependent modules. Encapsulating database access behind dedicated drivers prevents leakage of storage details into business logic, facilitating maintainability, unit testing, and microservice deployment.

3.2.4 Summary

The Component Diagram provides the physical software view of the platform. It establishes modular build artifacts, interface specifications, and database connectors required for implementation and containerized deployment.

4 Object-Oriented Static Design

Static structure modeling defines the object-oriented domain classes, encapsulated attributes, member methods, and structural associations that represent the core domain entities and data structures of the application.

4.1 Class Diagram Model

Figure 4.1 presents the Class Diagram of the Nepali Lifestyle Based Diet and Fitness Advisory System, capturing static class structures, attribute encodings, method signatures, and multiplicity relationships across domain entities.

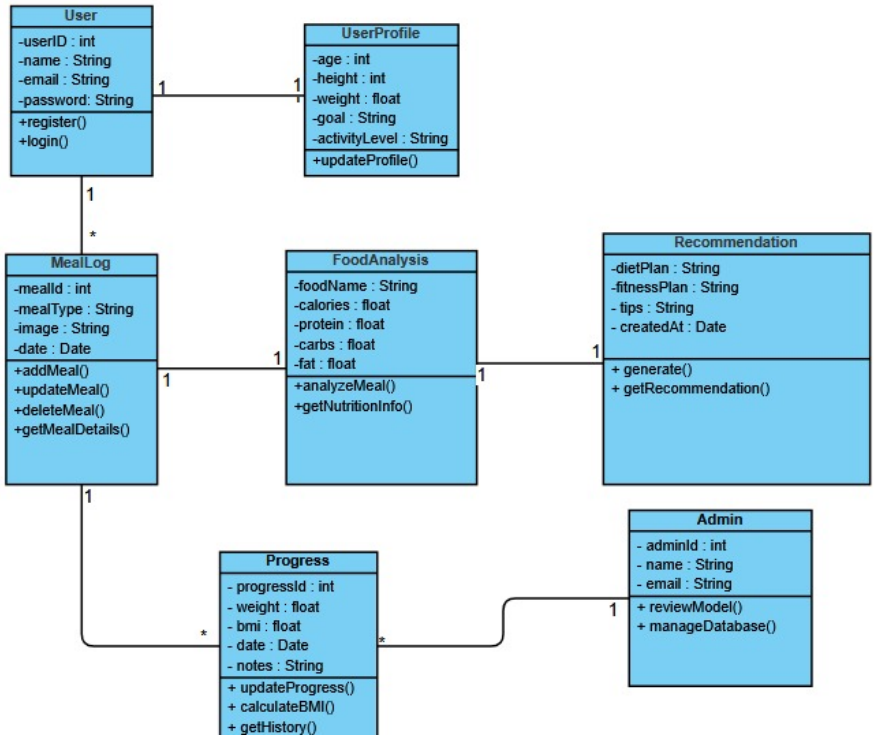


Figure 4.1: Class Diagram of the Advisory System

4.1.1 Major Classes and Attributes

The static domain structure comprises seven primary classes:

- **User**: Represents authenticated platform users. Encapsulates identity and credential attributes (`userID`, `name`, `email`, `password`) and exposes authentication operations (`register()`, `login()`).
- **UserProfile**: Encapsulates physical health and lifestyle parameters (`age`, `height`, `weight`, `goal`, `activityLevel`) along with profile update operations (`updateProfile()`).
- **MealLog**: Captures individual dietary logging events (`mealId`, `mealType`, `image`, `date`) and lifecycle management operations (`addMeal()`, `updateMeal()`, `deleteMeal()`, `getMealDetails(`

- **FoodAnalysis**: Models analytical food item processing and nutritional breakdown (`foodName`, `calories`, `protein`, `carbs`, `fat`) and provides computation routines (`analyzeMeal()`, `getNutritionInfo()`).
- **Recommendation**: Encapsulates synthesized health and dietary guidance (`dietPlan`, `fitnessPlan`, `tips`, `createdAt`) and generation routines (`generate()`, `getRecommendation()`).
- **Progress**: Tracks historical biometric trends and BMI calculations over time (`progressId`, `weight`, `bmi`, `date`, `notes`) via update and history retrieval operations (`updateProgress()`, `calculateBMI()`, `getHistory()`).
- **Admin**: Represents system administrators managing backend governance (`adminId`, `name`, `email`) with administrative operations (`reviewModel()`, `manageDatabase()`).

4.1.2 Relationships and Multiplicities

Structural associations define entity cardinalities and structural dependencies:

- **User to UserProfile**: A strict 1-to-1 association ($1 \leftrightarrow 1$) links each user to a unique health profile entity, ensuring clear separation between identity management and health metrics.
- **User to MealLog**: A 1-to-many association ($1 \leftrightarrow *$) indicates that a single user can log multiple dietary entries over time.
- **MealLog to FoodAnalysis**: A 1-to-1 association ($1 \leftrightarrow 1$) pairs each meal log entry with a specific nutritional analysis instance.
- **FoodAnalysis to Recommendation**: A 1-to-1 association ($1 \leftrightarrow 1$) links analyzed meal nutritional data to the resulting recommendation output.
- **MealLog to Progress**: A 1-to-many relationship ($1 \leftrightarrow *$) tracks cumulative meal log inputs against longitudinal progress updates.
- **Admin to Progress**: A 1-to-many relationship ($1 \leftrightarrow *$) enables administrative oversight and model performance validation across user progress records.

4.1.3 Design Discussion

The class design adheres strongly to Object-Oriented principles, notably Encapsulation and the Single Responsibility Principle (SRP). Attributes are private (-), exposing functionality exclusively through public operations (+). Separating *User* credentials from *UserProfile* health parameters isolates security and authentication concerns from biometric calculations. Similarly, decoupling *MealLog* event tracking from *FoodAnalysis* computational routines allows food recognition algorithms to evolve independently of client input modalities. This static class layout maps directly onto relational Object-Relational Mapping (ORM) entities in PostgreSQL and TypeScript data interfaces on the frontend.

4.1.4 Summary

The Class Diagram establishes the static structural blueprint for the codebase. It provides the object-oriented specification required for implementing backend data models, ORM persistent entities, and frontend type definitions.

5 Overall Discussion

The UML models developed in this laboratory report establish a comprehensive, multi-dimensional specification of the Nepali Lifestyle Based Diet and Fitness Advisory System. Each diagram addresses a specific architectural concern while maintaining total alignment with the Data Flow Diagrams produced in Lab 2.

The Architectural Block Diagram, Component Diagram, and Class Diagram establish the structural topology, dividing the application into five manageable tiers (Client, Gateway, Processing, Data, and External AI) and specifying domain entities. This organization directly supports non-functional requirements such as scalability, maintainability, and security. By isolating computationally demanding tasks—such as computer vision image processing in the YOLO Engine and vector embeddings in ChromaDB—from standard user profile management, individual services can scale horizontally based on real-time load.

Behaviorally, the Use Case, Activity, and Sequence Diagrams capture dynamic operational characteristics. The Use Case Diagram outlines functional boundaries and user roles, while the Activity Diagram details algorithmic decision paths and error recovery during meal logging. The Sequence Diagram unifies these views by demonstrating how authentication, food analysis, nutrient aggregation, database persistence, and LLM advice synthesis execute as a cohesive, time-ordered sequence of module calls.

From a software engineering perspective, the combined design adheres to established architectural principles:

- **Separation of Concerns**: Clean division between presentation, routing, business logic, persistence, and external AI services.
- **High Cohesion and Loose Coupling**: Microservices and domain classes perform tightly focused tasks and communicate through explicit API contracts and typed interfaces.
- **Safety and Constraint Enforcement**: Deterministic rule filtering guarantees that generative AI advisory outputs remain safe for users with underlying medical conditions.

Taken together, these design models bridge the gap between high-level requirements and physical implementation, providing an unambiguous blueprint for object-oriented class implementation and relational database schema construction.

6 Conclusion

In this laboratory experiment, complete structural, behavioral, and static object-oriented UML models were successfully designed for the Nepali Lifestyle Based Diet and Fitness Advisory System. Building upon functional requirements from Lab 1 and data flow architectures from Lab 2, this lab specified the system's runtime multi-tier architecture, use case boundaries, activity decision workflows, inter-module sequence messaging, component interface dependencies, and static class structures.

The resulting design demonstrates high modularity, robust error handling, and clear separation of concerns across client, gateway, processing, database, domain entity, and AI service layers. Completing this behavioral, structural, and class modeling provides a solid, engineering-grade foundation for the upcoming object-oriented programming implementation and database schema generation phases of the project.